Stato di Avanzamento del Progetto "AI-WAF"

Questo documento delinea lo stato attuale del progetto di tirocinio rispetto ai requisiti iniziali (Fasi 1, 2 e 3 del documento di progetto). L'obiettivo è evidenziare i moduli completati con successo e dettagliare le componenti attualmente parziali o mancanti che richiedono sviluppo futuro.

☑ Moduli Completati

Il nucleo dell'architettura "Agentic WAF" è stato implementato e funzionante:

1. **WAF Wrapper Intelligente (Fase 1)**:
   - L'agente (`autonomous_ctf_agent.py`) astrae la complessità di configurazione di Firegex.
   - Rilevamento automatico dei servizi vulnerabili tramite ispezione Docker.
2. **Infrastruttura MCP & API (Fase 2)**:
   - **Server MCP (`firegex_mcp_server.py`)**: Implementato pienamente lo standard *Model Context Protocol* per esporre le funzionalità del WAF come "Tools" (list_files, setup_waf, add_rule).
   - **Comunicazione Sicura**: L'agente comunica con il WAF tramite token e chiamate API strutturate.
3. **Core "Agentic" & Rule Suggestion (Fase 2)**:
   - **Analisi Ibrida**: Il modulo `AgentBrain` integra scansione statica (Bandit) e Generativa (LLM/Ollama) per individuare vulnerabilità nel codice sorgente.
   - **Proposta Regole**: Il sistema genera automaticamente regex difensive (es. SQL Injection, XSS) e le sottopone all'approvazione umana.

⚠ Componenti Parziali o Mancanti

Le seguenti aree rappresentano il divario attuale tra l'implementazione corrente e la visione completa del progetto (Fasi 1-3).

1. Clonazione del Traffico (Traffic Mirroring) automatizzata

- **Requisito (Fase 1)**: "*Configurazione WAF con clonazione traffico... Creazione dell'ambiente simulato in modo automatico*".
- **Stato Attuale**: **Concettuale / Manuale**.
  - Esiste documentazione teorica (`Explain/replayer/traffic_cloning.md`) e script di replay (`replayer.py`).
  - **Manca**: L'automazione a livello di infrastruttura (es. regole `iptables -j TEE` o configurazione Docker network driver) che duplichi trasparentemente ogni pacchetto in ingresso dalla VM Reale alla VM Simulata senza intervento manuale. Attualmente l'agente opera direttamente sul target rilevato.

2. Feedback Loop su SLA (Simulation Validation)

- **Requisito (Fase 3)**: "*Retrieving delle metriche relative alle rule... Integrazione con ambiente simulato*".
- **Stato Attuale**: **Non Implementato**.
  - L'agente sa applicare regole, ma non possiede ancora la logica di controllo retroattivo: "*Se applico questa regola, rompo il checker?*".
  - **Manca**: Il ciclo chiuso in cui l'agente applica la regola *prima* sull'ambiente Shadow, attende il passaggio del checker simulato, verifica lo stato del servizio (SLA), e solo in caso di esito positivo promuove la regola in produzione.

3. User Interface (UI)

- **Requisito (Fase 2)**: "*User interface per interazione e gestione utente*".
- **Stato Attuale**: **CLI Only**.
  - L'interazione avviene interamente tramite terminale (`autonomous_ctf_agent.py`).
  - **Manca**: Una dashboard Web (es. React/Vue) per visualizzare graficamente lo stato dei due ambienti (Reale vs Simulato), i log del traffico clonato e i suggerimenti dell'AI in tempo reale.

Note per la Consegna

Il codice sviluppato dimostra la fattibilità dell'approccio "Agent-based" per la sicurezza difensiva. L'agente è in grado di ragionare sul codice e agire sul firewall in autonomia. Le parti mancanti riguardano principalmente l'orchestrazione dell'infrastruttura di rete parallela (Simulazione), che rappresenta il naturale step successivo (Future Work) per rendere il sistema applicabile in una competizione A/D reale senza rischi per la SLA.